Practical Work No. 12
Recursivity

I. Definition and classification

Recursion is a very powerful tool in programming. When used correctly, it makes programming easier. Above all, it is a method for solving problems.

There are several types of recursion:

- **Direct recursion**: when a module calls itself.
- **Indirect or mutual recursion or Cross recursion**: when module A calls module B, which in turn calls A.
- **Circular recursion**: when module A calls module B, module B calls module C, and module C calls A.

Regarding methods, other classifications of recursion can be found:

- **Non-tail recursion**: A recursive method is said to be non-tail recursive if the result of the recursive call is used to perform additional processing (besides returning the module's result).
- **Tail recursion**: A recursive method is said to be tail recursive if no processing is performed when returning from a recursive call (except for returning the module's result)

II. Study of an Example: The Factorial Function

Denoted as **n!** (read as "n factorial"), it is the product of all positive integers from 1 to **n**, inclusive.

Examples: **4! = 1 × 2 × 3 × 4** **and** **5! = 1 × 2 × 3 × 4 × 5**

Note that **4!** can be written as: **4! = 4 × 3!**

Conclusion: **n! = 1** if **n = 0** or **n = 1**

n! = n × (n - 1)! **otherwise**

A. Non-Tail Recursive Solution

```
unsigned facto (unsigned n) {
  if (n == 1 || n == 0)
     return 1;
  else
     return n * facto(n - 1);
}
```

Illustration of the Execution Mechanism

`facto(4)` returns **4 × facto(3)**
`facto(3)` returns **3 × facto(2)**
`facto(2)` returns **2 × facto(1)**
`facto(1)` returns **1** (base case, recursion stops)
`facto(2)` returns **2 × 1 = 2**
`facto(3)` returns **3 × 2 = 6**
`facto(4)` returns **4 × 6 = 24**

Thus, **facto(4) = 24**.

B. Tail Recursive Solution

In a **tail-recursive** approach, the recursive call is the last operation performed in the function, allowing the compiler to optimize the recursion (tail-call optimization).

```
unsigned facto (unsigned n, unsigned result) {

  if (n == 0 || n == 1)
     return result;
  else
     return facto(n - 1, n * result);
}
```

III. Guidelines for Writing a Recursive Function

These guidelines are illustrated with the following example:

Write a recursive function to compute the sum of the digits of a positive integer n.

Example: For n=528, the sum of its digits is: 15

a) Parameterizing the Problem

Identify the elements that determine the solution and define the size of the problem.

- Here, the solution depends on the integer n, which we progressively reduce by removing its last digit.

b) Defining the Base Case

Determine the condition under which the solution is immediately known (trivial case).

- If given the choice between n=528 and n=8, we pick n=8, since the sum of its digits is trivially **8**.

Conclusion: If n is a **single-digit number**, we stop.

➔ **The sum of the digits is simply n itself**

c) Reducing the Problem to a Simpler Case

To ensure that the base case is reached at some point, we break the problem into smaller subproblems.

Using the example **528**, we decompose it as follows:

sumDigits(528)=8+sumDigits(52) = 8+(2+sumDigits(5)) =8+(2+5)=**15**

Each recursive step extracts the last digit (n mod 10) and reduces the problem by removing this digit (n div10).

d) Writing the Recursive Function

```
unsigned sumDigits(unsigned n) {
   if (n < 10)
      return n;
   else
      return (n % 10) + sumDigits(n / 10);
}
```

Exercise:
Illustrate the previous guidelines to write a recursive function that calculates the product of two positive integers **a** and **b** without using the multiplication operator (*).

Solution:
a) The solution to this problem depends on the two operands **n1** and **n2**.
b) If you have the choice between: 12 x 456, 12 x 0, 12 x 1, which of the three calculations is the simplest?
c) 12 x 9 = 12 + 12 + 12 + ... + 12 (9 times)
= 12 + (12 + 12 + ... + 12) (8 times)
= 12 + 12 x 8
and so on...

IV. Application Exercises

Exercise 1: Simple Recursion

Given the following numerical sequence U_n:

- If n=0, then $U_0=4$
- Otherwise, if n>0, then $U_n=2\times U_{n-1}+9$

Write a C function to calculate the term U_n.

Exercise 2: Cross Recursion

Write two C functions to calculate the first **n** terms (with **n** passed as an argument) of the integer sequences U_n and V_n defined below.

- U0=1 V0=0
- Un=Vn−1+1 Vn=2×Un−1

Exercise 3:

The Fibonacci sequence is defined as follows:

```
Fib(0) = 1
Fib(1) = 1
Fib(n) = Fib(n-1) + Fib(n-2)
```

Propose a non-tail recursive solution and a tail-recursive solution to calculate the n^{th} term of the Fibonacci sequence.

Exercise 4:

He wants to implement a function to calculate the GCD (PGCD in Fr) of two non-zero natural numbers (a and b) using Euclid's algorithm.

- Propose an iterative version
- Propose a non-tail recursive version
- Propose a tail-recursive version

```
                    a                          if  a=b
PGCD (a,b) =        PGCD (a-b , b)             if  a>b
                    PGCD (a , b-a)             if  b>a
```

Exercise 5:

Given an array `T` containing `n` elements of type `int`, implement the following **recursive** functions in C:

```
void fillArray(int *T, int n);
void displayArray(int *T, int n);
void reverseArray(int *T, int n);
int frequency(int *T, int n, int x);
/*The last function returns the nbr of occurrences of x in the array T.*/
```